

ChatGPT ▾

这是已分享的 ChatGPT 对话副本
举报对话

codex 并发插件，现在单文件工作效率太低了。有没有办法提高 codex工作效率

可以，而且现在很多人在给 Codex CLI 外挂“并发调度层”，因为原生单 session 确实容易卡在：

- 一个任务占满上下文
- 文件互相覆盖
- 长任务阻塞
- 多 repo / 多 feature 无法同时推进

目前提升 Codex 工作效率，核心思路其实就 4 个：

1. 用 git worktree + tmux 真并发（最有效）

这是现在社区主流方案。 [frr.dev +2](#)

原理：

- 每个 agent 在独立 worktree
- 每个 worktree 一个 branch
- tmux 保持多个 agent 同时运行
- 最后 merge

这样 Codex 不会互相覆盖文件。

基础结构

Bash

```
repo/  
repo-worktrees/  
├─ feat-login/  
├─ api-refactor/  
└─ tests/
```

开三个 Codex

Bash

```
git worktree add ../repo-worktrees/feat-login -b feat-login
git worktree add ../repo-worktrees/api-refactor -b api-refactor
git worktree add ../repo-worktrees/tests -b tests
```

然后：

Bash

```
cd ../repo-worktrees/feat-login && codex
cd ../repo-worktrees/api-refactor && codex
cd ../repo-worktrees/tests && codex
```

配合 tmux：

Bash

```
tmux new -s agents
```

直接 3~6 个 agent 并发。

2. 上 orchestration 工具（效率暴涨）

现在已经有人把 Codex 并发封装好了。

推荐一：dmux

这是目前最成熟的。 [dmux.ai](https://github.com/dmux/agents)

特点：

- 自动 worktree
- 自动 branch
- 多 agent A/B 测试
- tmux pane 管理
- merge 辅助
- 支持 Codex / Claude / Gemini 混跑

例如：

Bash

```
dmux
```

然后输入：

1. 修 auth
2. 补测试
3. 重构 api

它会自动：

- 建 branch
- 开 pane
- 分配 agent
- 隔离上下文

这个比裸 codex 强太多。

推荐二：codex-yolo GitHub

专门给 Codex CLI 做的并发层。 [GitHub](#)

支持：

- parallel agents
- auto approval
- worktree isolation
- merge conflict detection

例如：

Bash

```
codex-yolo -w \  
  "修登录 bug" \  
  "补 e2e" \  
  "更新 README"
```

它直接帮你开多个 agent。

缺点：

- 很激进
- 默认偏 YOLO 模式
- 小心 destructive command

适合个人项目。

3. 不要让一个 Codex 干“大任务”

这是很多人最大的性能瓶颈。

错误方式：

帮我重构整个支付系统

正确方式：

Agent1:

只负责 schema migration

Agent2:

只负责 API

Agent3:

只负责测试

Agent4:

只负责前端适配

LLM 的并发效率来自：

- 小上下文
- 小职责
- 小 diff

不是来自单 agent 超长上下文。

4. 建立“总控 agent + worker agents”

这是现在高阶玩法。 Reddit +1

架构：

```
graph TD
    Main[Main Codex] --> Queue[Task Queue]
    Queue --> Worker[Worker A/B/C]
```

主 agent：

- 规划
- 拆任务
- review PR
- merge

worker:

- 只改代码
- 只跑测试
- 只修 lint

这样 token 消耗会低很多。

因为：

主 session 不需要携带所有执行日志。

实际生产里推荐配置（非常稳）

我建议你直接这样：

轻量方案（最好落地）

tmux + git worktree + 3 个 codex

角色：

Agent	职责
A	feature
B	test
C	refactor/review

已经能提升 3~5 倍效率。

进阶方案

dmux + Codex + Claude

典型组合：

Agent	模型
Planner	Claude
Implementer	Codex

Agent	模型
Reviewer	Claude
Test Fixer	Codex

这是目前很多 AI coding heavy user 的玩法。 [dmux.ai](#) +1

Codex 当前最大瓶颈（很关键）

Codex 原生现在还没有成熟：

- worktree orchestration
- durable multi-agent memory
- conflict scheduling
- dependency graph planning

社区已经在补。 [frr.dev](#) +1

所以：

“不要把 Codex 当 IDE 插件”
要把它当“可调度 worker”。

效率会完全不同。

我最推荐的实际 workflow

适合中大型项目：

```
main/
└── orchestrator.md
```

```
worktrees/
├── feat-auth/
├── feat-api/
└── feat-tests/
```

tmux：



api	tests
-----	-------

流程：

- 1. planner 拆任务
- 2. worker 并发
- 3. 每个 worker 独立 branch
- 4. planner review
- 5. merge

这个会比单 Codex 快非常多。

 来源

我最推荐的是：

「git worktree + tmux + 多 Codex Agent」

这是目前最稳定、成本最低、最不容易翻车的方案。

比各种“自动 multi-agent 框架”更重要的是：

先把“隔离”做好。

因为 Codex 最大问题不是“不够聪明”，而是：

- 上下文污染
- 文件覆盖
- diff 冲突
- 长 session 退化
- 一个 agent 什么都干

而 worktree 天然解决这些问题。

一、整体架构（核心思想）

你不要再：

- 一个 codex
- 一个 session
- 一个超长任务

而应该：

一个职责 = 一个 agent = 一个 worktree

例如：

Agent A → API
Agent B → 前端
Agent C → 测试
Agent D → review

每个 agent：

- 独立目录
- 独立 branch
- 独立上下文
- 独立 terminal

这样 Codex 才会稳定。

二、目录结构（推荐）

推荐这样：

```
project/  
├─ main-repo/  
└─ worktrees/  
    ├─ feat-auth/  
    ├─ feat-api/  
    ├─ feat-tests/  
    └─ feat-review/
```

其中：

main-repo

只做：

- merge
- 最终 review
- orchestrator

不直接开发。

三、为什么 worktree 比 clone 强

很多人会：

Bash

```
git clone repo repo2  
git clone repo repo3
```

这是错误的。

因为：

- 占空间
- branch 不同步
- fetch 混乱
- 容易 merge 错

正确是：

Bash

```
git worktree
```

它共享：

- git object
- history
- refs

但工作目录隔离。

这是做并发 agent 的关键。

四、标准初始化（推荐照抄）

Step 1

主仓库：

Bash

```
mkdir project  
cd project  
  
git clone git@github.com:xxx/app.git main-repo
```

Step 2

创建 worktrees:

Bash

```
cd main-repo

git worktree add ../worktrees/feat-auth -b feat-auth
git worktree add ../worktrees/feat-api -b feat-api
git worktree add ../worktrees/feat-tests -b feat-tests
```

现在:

```
feat-auth
feat-api
feat-tests
```

都是独立 workspace。

五、tmux 才是灵魂

很多人忽略 tmux。

实际上:

tmux 才是 agent orchestration 的核心。

推荐布局:

Planner	
Auth	API
Tests	Review

六、每个 pane 一个 Codex

例如:

Pane 1

Bash

```
cd worktrees/feat-auth  
codex
```

Prompt:

你只负责 auth 模块。
禁止修改其他模块。
完成后提交 commit。

Pane 2

Bash

```
cd worktrees/feat-api  
codex
```

Prompt:

你只负责 API 重构。
不要碰 auth。

Pane 3

Bash

```
cd worktrees/feat-tests  
codex
```

Prompt:

你只负责：

- 测试
- lint
- typecheck

这样稳定性会暴增。

七、最关键：职责隔离

这是效率暴涨的真正原因。

很多人以为：

“并发”提升效率。

其实不是。

真正提升的是：

“上下文变小”

例如：

错误方式：

```
帮我重构整个系统
```

上下文：

- 500 文件
- 30k token
- 巨型 diff

Codex 会迅速退化。

正确方式：

```
Auth agent:  
只看 src/auth
```

```
API agent:  
只看 src/api
```

```
Test agent:  
只看 tests/
```

上下文：

- 小
- 干净
- 稳定

Codex 输出质量会高很多。

八、Planner Agent（最重要）

真正高级玩法：

一个 planner + 多 worker

Planner 不写代码。

Planner 只：

- 拆任务
- 分配任务
- review
- merge
- 生成 TODO

例如：

你是 orchestrator。

任务：

1. 拆分 feature
2. 给 worker 下发任务
3. 检查冲突
4. merge 结果

它相当于：

Tech Lead AI

九、推荐的 Prompt 模板（非常重要）

Worker Prompt

建议固定模板：

你是 worker agent。

规则：

- 只能修改指定目录
- 不允许大规模重构
- 不允许改配置
- 每次修改后运行测试
- 输出 summary
- commit message 必须规范

这会减少大量灾难。

Review Agent

你只负责 review:

- bug
- 类型错误
- architecture 问题
- 性能问题

禁止直接修改代码。

这个很好用。

十、真正高效的目录切分

不要按“功能”切。

应该按：

“冲突概率”

错误：

Agent A:
用户系统

Agent B:
支付系统

但两者都改：

```
routes.ts
```

会冲突。

正确：

Agent A:
backend only

Agent B:
frontend only

Agent C:
tests only

冲突最小。

十一、如何避免 Codex 越跑越傻

这是很多人的痛点。

原因：

- session 太长
- token 污染
- 历史 diff 太多
- prompt 漂移

解决：

不要连续聊 2 小时

建议：

30~40 分钟
直接重开 session

用 AGENTS.md

例如：

```
project/  
  AGENTS.md
```

写：

Markdown

Rules

- 使用 pnpm
- 禁止改 ESLint

- 禁止改 tsconfig
- 必须跑测试

所有 agent 自动继承。

这个效果非常大。

十二、最佳实践（生产可用）

我现在最推荐：

小项目

2~3 agents

够了。

中项目

Planner
Frontend
Backend
Tests
Review

5 agents 最舒服。

大项目

开始需要：

- task queue
- auto merge
- dependency graph
- PR automation

这时再考虑：

- dmux
- OpenHands
- Claude Code orchestration
- custom scripts

十三、为什么这个方案最推荐

因为它：

方案	稳定性
单 codex 长 session	差
clone 多仓库	很乱
自动 multi-agent 框架	容易失控
worktree + tmux	最稳

这是目前：

“人类还能控制”的最佳并发方案。

十四、最终推荐配置（真的很好用）

我的建议：

必装

- tmux
- git worktree
- lazygit
- fd
- ripgrep

Agent 数量

推荐：

3~5 个

超过 8 个：

你会开始管理 agent，而不是开发。

十五、你下一步应该怎么做

直接试这个：

第一天

先开：

```
2 agents
```

例如：

```
Agent A → feature
```

```
Agent B → tests
```

感受一下。

第二天

加：

```
Planner Agent
```

开始 orchestrate。

第三天

再尝试：

- 自动 merge
- auto review
- 自动 commit message

这时候你会发现：

Codex 不再是“聊天机器人”。

而是：

“可并发调度的软件工程 worker”。

git worktree 是什么意思，这个怎么用的？

git worktree 是什么

git worktree 可以让：

一个 Git 仓库，同时拥有多个“工作目录（workspace）”。

最关键的是：

多个目录
共享同一个 .git

所以它比：

Bash

```
git clone
```

轻很多，也专业很多。

先理解 Git 的本质

普通情况下：

```
repo/  
├── .git  
└── 代码文件
```

你一次只能：

- checkout 一个 branch
- 在一个目录工作

例如：

Bash

```
git checkout feature-a
```

那你就不能同时在：

```
feature-b
```

开发。

worktree 解决什么问题

它允许：

同一个仓库
同时 checkout 多个 branch

例如：

main 分支 → 主目录
feature-a → worktree A
feature-b → worktree B

你会得到：

```
project/  
├─ main-repo/  
├─ feat-a/  
└─ feat-b/
```

它们：

- 都是同一个 Git 仓库
- 但可以同时开发不同 branch

这就是 AI agent 并发的核心。

为什么 AI coding 特别适合 worktree

因为：

每个 Codex agent：

都需要独立上下文

否则会：

- 改同一个文件
- 上下文污染
- branch 混乱
- diff 覆盖

所以：

一个 agent
一个 worktree

是最自然的结构。

最简单例子（建议你实际试）

Step 1: 创建仓库

Bash

```
mkdir demo
cd demo

git init
```

建个文件：

Bash

```
echo "hello" > app.txt
git add .
git commit -m "init"
```

Step 2: 创建 worktree

现在：

Bash

```
git worktree add ../feature-login -b feature-login
```

意思：

在 ../feature-login
创建一个新的工作目录
并新建 branch: feature-login

现在目录变成：

```
demo/
feature-login/
```

Step 3: 进入新 worktree

Bash

```
cd ../feature-login
```

你会发现：

Bash

```
git branch
```

当前 branch：

```
feature-login
```

而原目录：

```
demo/
```

还是：

```
main
```

这时发生了什么（很重要）

你现在：

```
两个目录  
两个 branch  
同时存在
```

并且：

```
互不影响
```

这就是它厉害的地方。

试试同时开发

主目录

Bash

```
cd demo
echo "main change" >> app.txt
```

worktree

Bash

```
cd ../feature-login
echo "feature change" >> app.txt
```

两个 branch:

- 同时工作
- 同时存在

这就是并发开发。

查看所有 worktree

Bash

```
git worktree list
```

会看到:

/path/demo	abc123 [main]
/path/feature-login	def456 [feature-login]

删除 worktree

例如:

Bash

```
git worktree remove ../feature-login
```

注意:

这只删除工作目录。

branch 还在。

删除 branch

Bash

```
git branch -D feature-login
```

为什么它比 clone 好

clone 的问题

如果你：

Bash

```
git clone repo repo2
```

```
git clone repo repo3
```

会：

问题 1：磁盘浪费

每个 clone：

- 都复制整个 git object
- history
- refs

很大。

问题 2：fetch 不同步

你需要：

Bash

```
git fetch
```

很多次。

问题 3: branch 混乱

容易:

哪个 repo 改了什么?

不清楚。

worktree 的优势

它:

共享 Git 数据
只隔离工作目录

所以:

- 快
- 轻
- 专业
- 稳定

AI Agent 为什么必须 worktree

因为:

AI 最怕“共享上下文”

例如:

两个 Codex:

都在同一个目录

会:

- 覆盖文件
- 误删代码
- checkout branch 冲突
- stash 混乱

而 worktree:

天然隔离

所以：

一个 agent = 一个 worktree

是黄金结构。

实际项目推荐结构

例如：

```
myapp/  
├─ main/  
└─ worktrees/  
    ├─ auth/  
    ├─ api/  
    ├─ tests/  
    └─ review/
```

创建：

Bash

```
cd main  
  
git worktree add ../worktrees/auth -b feat-auth  
git worktree add ../worktrees/api -b feat-api  
git worktree add ../worktrees/tests -b feat-tests
```

每个目录跑一个 Codex

例如：

Terminal 1

Bash

```
cd worktrees/auth  
codex
```

Terminal 2

Bash

```
cd worktrees/api  
codex
```

Terminal 3

Bash

```
cd worktrees/tests  
codex
```

现在：

3 个 AI
同时开发

而且：

互不干扰

最后怎么合并

每个 agent：

Bash

```
git commit
```

之后：

回主仓库：

Bash

```
cd main
```

merge：

Bash

```
git merge feat-auth  
git merge feat-api  
git merge feat-tests
```

一个非常重要的理解

worktree 不是：

复制仓库

而是：

“同一个仓库的多个平行工作空间”

这个概念非常重要。

你现在最应该做的

建议你：

先别搞复杂 orchestration

先实际试：

Bash

```
git worktree add ../test-agent -b test-agent
```

然后：

Bash

```
cd ../test-agent  
codex
```

你会瞬间理解：

为什么 AI coding 的并发核心其实是 worktree。

明白。我先使用 codex plan 模式，然后计划好我的并发和计划。然后让他把 todo.md 和 agents.md 分别写到分层好的 git worktree 工作目录中，在使用 dmux 并发执行。这个思路对吗？

对，而且这是目前非常“工程化”的 AI coding workflow 了。

你这个思路已经接近真正的 multi-agent software pipeline：

```
Codex(plan)
  ↓
任务拆分
  ↓
生成 worktrees
  ↓
生成 agents.md / todo.md
  ↓
dmux 并发执行
  ↓
review / merge
```

这个方向是对的。

但我建议你再升级一点点结构，否则后面 agent 一多会乱。

我建议的完整架构（非常稳）

推荐：

```
main-repo/
├── orchestration/
│   ├── master-plan.md
│   ├── dependency-graph.md
│   └── merge-order.md
├── worktrees/
│   ├── auth/
│   │   ├── AGENTS.md
│   │   └── TODO.md
│   └── api/
│       ├── AGENTS.md
│       └── TODO.md
```



这里最关键的是：

orchestration 层

很多人没有这一层。

结果：

- agent 不知道依赖关系
- merge 顺序混乱
- 多 agent 改同一文件
- planner 上下文丢失

你现在这个流程里，最正确的部分

就是：

先 plan，再并发

这是对的。

不要：

直接开 5 个 codex 开干

否则一定会：

- 文件冲突
- architecture 漂移
- agent 做重复工作

推荐 workflow（生产级）

Step 1: 先让 Codex 只做 Planner

非常重要：

planner 不写代码

只做：

- task decomposition
- dependency graph
- conflict analysis
- branch strategy

例如：

你是系统 orchestrator。

请：

1. 分析 feature
2. 拆成独立 worker
3. 标记文件冲突风险
4. 设计 merge 顺序
5. 输出 worktree 结构

这一阶段：

禁止修改代码

非常重要。

Step 2: 生成 master-plan.md

例如：

Markdown

```
# Master Plan
```

```
## Agent Auth
```

目录：

- src/auth
- middleware/auth

禁止：

- routes/*
- database/*

依赖：

- api-agent

```
---  
  
## Agent API
```

```
目录：  
- routes/*  
- controllers/*
```

这个：

才是整个系统的大脑。

Step 3: 自动生成 worktree

例如脚本：

```
Bash  
  
#!/bin/bash  
  
agents=("auth" "api" "tests")  
  
for agent in "${agents[@]}"  
do  
    git worktree add ../worktrees/$agent -b feat-$agent  
done
```

Step 4: 生成 AGENTS.md（关键）

很多人低估这个。

实际上：

AGENTS.md 是 agent 的 system prompt。

例如：

```
Markdown  
  
# AGENT RULES  
  
你只负责 auth 模块。  
  
禁止：  
- 修改 package.json  
- 修改 tsconfig
```

- 修改 routes

必须：

- 运行测试
- 使用 pnpm
- 输出 summary

这个能大幅降低 AI 漂移。

Step 5: 生成 TODO.md (关键)

例如：

Markdown

TODO

- [] 添加 JWT middleware
- [] 添加 refresh token
- [] 更新 auth tests
- [] 添加 types

注意：

TODO 要“小颗粒度”

不要：

- 重构认证系统

而是：

- 添加 middleware
- 更新 schema
- 增加 test

因为：

小 TODO = 小 context = 高质量

Step 6: dmux 并发执行

这是正确时机。

不要在：

没有 plan
没有 AGENTS
没有 TODO

时启动 dmux。

否则 agent 很快会：

- 漂移
- overlap
- context explosion

Step 7: Review Agent（非常重要）

建议：

单独一个 review worktree

例如：

```
worktrees/review
```

它只负责：

- review diff
- architecture consistency
- test validation
- merge readiness

不要让 worker 自 review。

最重要的一点：依赖图

你已经开始接近真正的大规模 AI coding。

所以：

dependency graph 会非常重要。

例如：

```
auth-agent  
↓  
api-agent
```

↓
frontend-agent

如果：

frontend
依赖 api schema

那：

frontend agent 必须后启动。

否则会：

- context mismatch
- stale API
- hallucinated interfaces

你现在最容易踩的坑

坑 1：多个 agent 改同一个文件

例如：

routes.ts

解决：

planner 必须输出：

FILE OWNERSHIP

例如：

Markdown
routes/*
owner: api-agent

这个特别重要。

坑 2: Agent 上下文失控

解决:

强制:

一个 agent 只看一个子目录

不要:

给整个 repo

坑 3: TODO 太大

错误:

实现支付系统

正确:

- 创建 payment schema
- 创建 payment service
- 创建 payment route
- 增加 payment tests

坑 4: merge chaos

解决:

planner 输出:

Markdown

MERGE ORDER

1. auth
2. api
3. frontend
4. tests

这个很重要。

我最推荐你加的一层（非常值）

你现在已经：

```
Codex Plan  
→ dmux
```

下一步我建议：

加一个“review gate”

例如：

```
worker  
  ↓  
review-agent  
  ↓  
merge
```

review-agent 只做：

- architecture consistency
- duplicated logic
- bad abstractions
- lint
- types
- security

这个会让系统稳定很多。

最后给你一个真正生产可用的结构

推荐最终：

```
Planner Agent  
  ↓  
Generate Worktrees  
  ↓  
Generate AGENTS.md  
  ↓  
Generate TODO.md  
  ↓  
dmux Parallel Workers  
  ↓  
Review Agent  
  ↓  
Merge Agent
```

这是现在非常先进的 AI coding workflow 了。

你已经走在正确方向。